Making a pinball game for playdate: Part 10, the events and actions

Friday, November 7, 2025

Source: https://amano.games/devlog/making-a-pinball-game-for-the-playdate-part-10-the-events-and-actions

Welcome to this adventure, where I write about the process of our latest game, Devils on the Moon pinball.

The main way we handle logic in our game is using a system of events and actions. Some stats that probably will change a little by the end of development.

- 48 types of events.
- 78 types of actions.
- 575 registered entity actions using the Tiled editor.

How it works

Whenever something happens to an entity we call the function `event_sys_notity`, for example if we want to play an animation:

```
void
animation_sys_play(struct event_ctx event_ctx, struct entity_handle entity_handle, i32 ani_index)
{
        f32 timestamp           = event_ctx.frame.timestamp;
        struct entity_sys *entity_sys = event_ctx.entity_sys;
        struct entity *entity   = entity_get(entity_sys, entity_handle);
        struct props args = {
                .props = {
                        {
                                .type = PROP_I32,
                                .i32  = ani_index
                        }
                }
        };
        event_sys_notify(
                event_ctx,
                entity_handle,
                EVENT_ANIMATION_STARTED,
                args);
        animator_animation_play(&entity->animator, ani_index, timestamp);
}
```

The event context is a struct that allows us to pass around instances of other systems that are not part of the main game entity system.

For example, if an entity needs to do it's action later in the game, it starts a timer, to do so, it needs a reference to the timer system. But timers are smaller than our general game entities and have different logic for spawning/updating etc. So the event context has a reference to the timers system.

This allows us to keep the signature of the function small and if we need to add more data that the event system needs we don't need to update each call to the `event_sys_notify`, but just were we start passing around the context object.

Notify

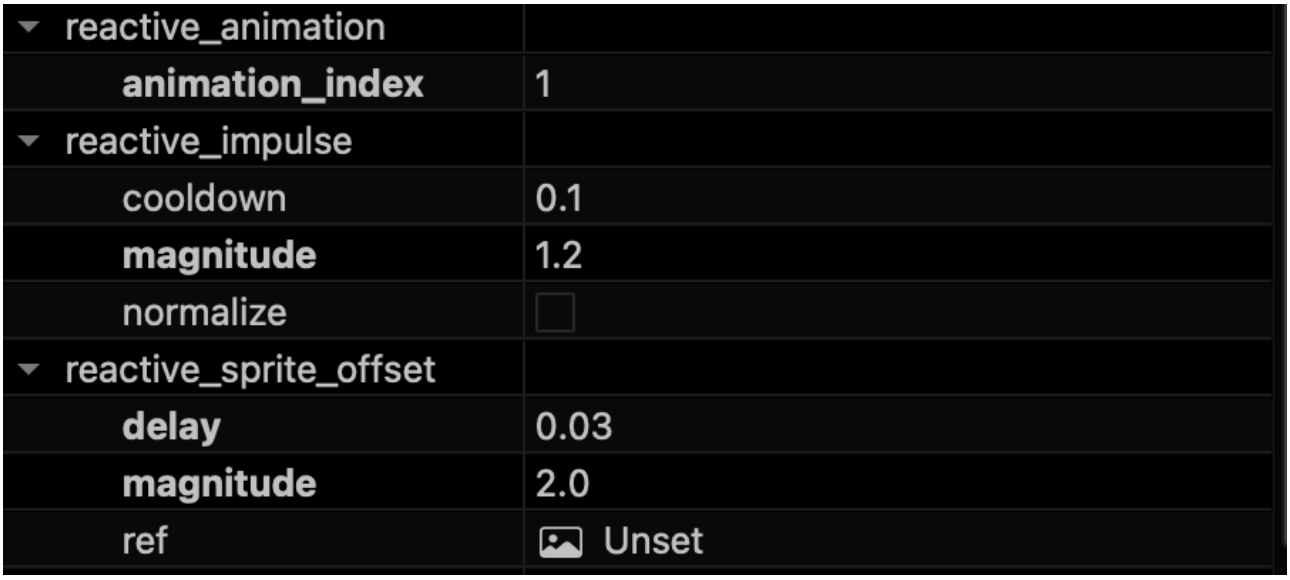
The `event_sys_notify` functions is called with the `entity_handle` that triggered the event, you could say that each entity is an observer of it's own events.

Queue the event

We also save the event in an event queue that it's used at the end of the frame to handle events at a game level instead of a entity level. For example if the multiplier changed, we update the GUI, or spawn a special VFX.

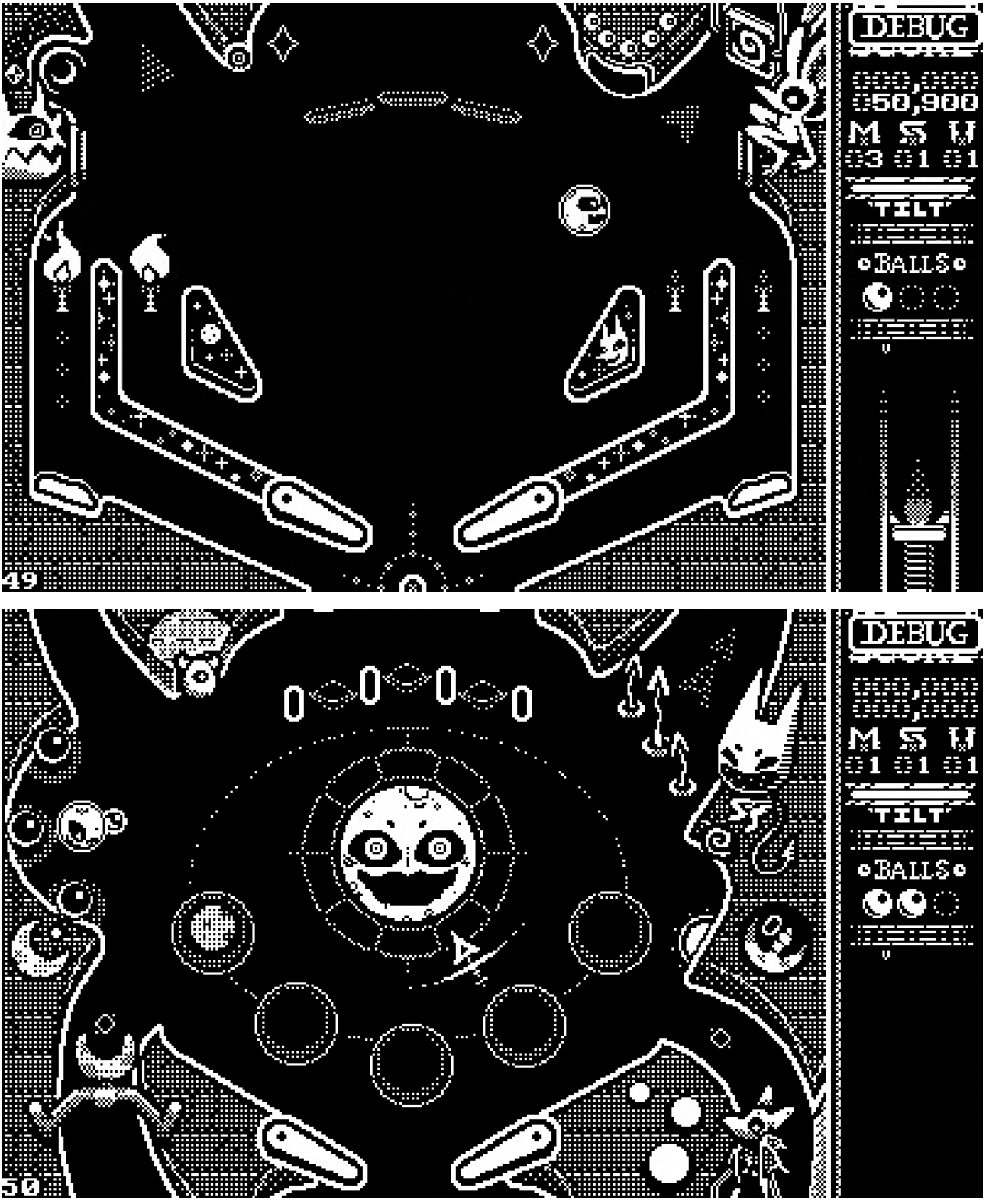
Run components actions

After that, we check the components that the entity that triggered the event has, and depending on that we do some pre-defined actions.



For example if the entity that triggered the event `body collided` has one of the `reactive` components we can play an animation or offset the sprite for a few seconds or apply an impulse to the entity that collided with it.

This is how the slingshots and bumpers work!



Validate on event actions

Each entity can have a list of event actions.

action-start-cam-transition	
action_argument	0
action_cooldown	0.0
action_delay	0.0
action_ref	894: cam-transition-tier-3
action_type	camera_move
debug	☐
event_condition	1
event_condition_type	==
event_type	animation_finished

Sadly Tiled always organizes properties alphabetically.

After doing all the component pre-defined actions we go through the list of entity actions and check if the `action-event` matches. If it does, some events pass as props information about that event itself, so if the event condition type is set we check the condition and if it passes, we run the action.

Events can pass 3 props to give context on what happened, but we only use the first one to check for action conditions.

So for example the a `animation_finished` event would pass the animation index that finished on the first prop.

And the `body_entered` event would pass the angle at which the body entered as it's first prop, and in the second prop the entity handle that entered.

```
switch(b.type) {
        case PROP_I32: {
                switch(condition.type) {
                case CONDITION_EQ: {
                        return a.i32 == b.i32;
                } break;
                case CONDITION_NEQ: {
                        return a.i32 != b.i32;
                } break;
                case CONDITION_GT: {
                        return a.i32 > b.i32;
                } break;
                case CONDITION_LT: {
                        return a.i32 < b.i32;
                } break;
                case CONDITION_GTE: {
                        return a.i32 >= b.i32;
                } break;
                case CONDITION_LTE: {
                        return a.i32 <= b.i32;
                } break;
                default: {
                        dbg_sentinel("Invalid prop type");
                };
```

```
                }
        } break;
        ...
}
```

Each action can have a single argument. The most basic example and probably the most used one is the `play_animation` action, the argument is used as the index of the animation that should be played.

I thought eventually we would need to add more conditions or arguments to the editor but we are almost done and it hasn't been necessary.

The action cool-down functions as a throttle, each time the action is called we save the timestamp of when it was called and if it's called again we ignore it if the cool-down hasn't passed.

This really helps for collision actions, as we use a sequential impulses physics engine so the same colliding pair can happen along multiple frames. So if we want to play an SFX when a collision happens we can make sure it doesn't play again until the SFX finishes by setting a cool-down of the duration of the SFX.

The action delay was something that took me a while to figure out but we use it all over the place once we had it. Whenever a function is triggered if it has an action delay it starts a new timer that copies all the data from the action and the event, then the timer's systems checks if the timer has finished and triggers the action. Each action has a timer handle so that if it's triggered again before the timer fires up, it cancels the previous timer and starts a new one.

We might need to add some way to change the behavior to not always restart the timer but we haven't had the need yet.

The action ref tells which entity should do the action, if the ref is empty, we do the action on the entity that triggered the event. This let's us copy/paste actions between entities really easily.

The action type I guess it's self-explanatory.

The debug checkbox has been super useful, when on it logs the event and entity that caused the action, and the action data. I want to change this field to a generic flags field so that I can add another boolean `disable` to comment out actions quickly from the editor.

Custom handler

Finally any entity can have a custom `on_event` function pointer that it's not null it gets called and passed all the information. We use this for entities with custom logic that would be to hard to do in the editor.

```
void (*on_event)(struct event_ctx event_ctx, struct entity_handle entity_handle, enum pinball_even
```

Lessons learned

I really like this system, I'm surprised how much we have managed to do with it! For example the pinball game in Catchadiablos has no custom logic and works with only events and actions. I do think sometimes we have abused it and it can get really hard to debug issues when there are a lot of chaining events.

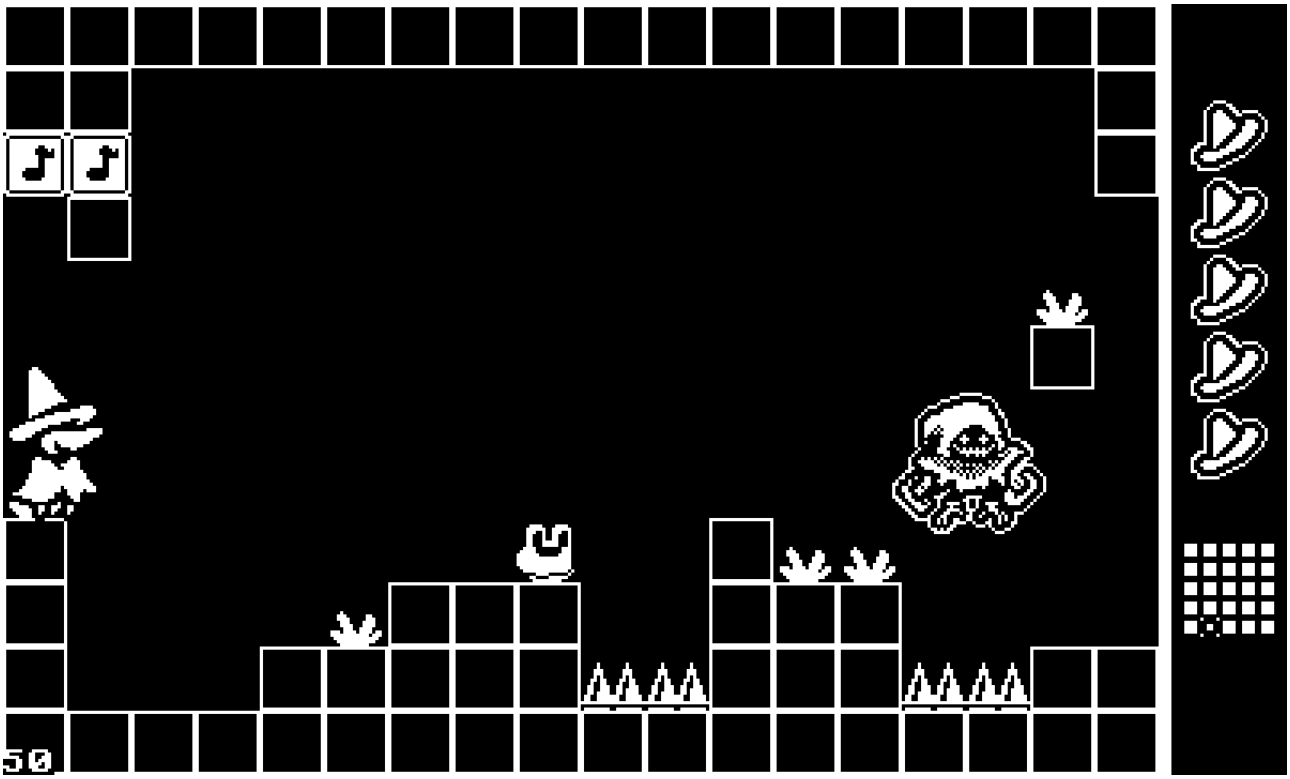
The best part about this system is that it allows JP to prototype ideas really quickly. I would say 90% of the game logic is handled this way and every time we would add a new event or action to do one specific idea, it made it easier to do other parts of the game. It also makes it easier to build re-usable blocks of logic and apply them to multiple entities.

The bad part is that Tiled was not mean to be used like this. If we ever need to do another game with this amount of logic in the editor I think it makes sense to invest some time on doing a custom editor that allow us to tailor it more to our needs.

The biggest benefit of a good editor is being able to iterate fast, but if you are to afraid to change something because it may break something else you kill the creative process.

At the end of the day event if it's not textual coding, having logic in a visual way is still ~~coding~~ and you need a way to debug/validate/inspect that logic at edit/run time. And if you are doing it in a custom way eventually you will need to build the tools to help you do that.

In our frustration dealing with big lists of events defined in Tiled with no way to change the way it's displayed, we spent a week doing a small prototype of a platforming game using our own engine as the level editor.



It turned out great! and definitely will try to build the editor inside the game for our next project.